

СХЕМЫ ДАННЫХ (DATA LAYER) — V1.0 COMPLETE

Дата: 2026-06-26 | **Версия:** 2.0 (Production Ready)

Автор: Koda (AI) + NLP-Core-Team

Назначение: Полная структура данных для реализации всех функций первой версии.

1. SERVER-SIDE (PostgreSQL + Drizzle ORM)

Хранение: Relational DB (PostgreSQL 16+).

ORM: Drizzle ORM — TypeScript-native, light-weight, генерирует миграции из schema.

Принцип: Type-safe queries без ORM-заглушек. Совместимость с CockroachDB/YugabyteDB.

Политика Удаления: Все данные удаляются мягко (`deleted_at`). Физическое удаление через 10 лет (комплаенс).

1.1 Таблицы (DDL)

 Аутентификация и Логи

```
-- auth_logs.sql (История входов)
CREATE TABLE auth_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,

  ip_address INET NOT NULL,
  user_agent TEXT,
  location_json JSONB, -- { country, city }

  success BOOLEAN DEFAULT FALSE,
  failure_reason VARCHAR(100), -- WRONG_PASSWORD, INVALID_CODE

  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_auth_logs_user ON auth_logs(user_id);
CREATE INDEX idx_auth_logs_created ON auth_logs(created_at DESC);
```

 Пользователи и Контакты

```
-- users.sql
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

-- Auth
email VARCHAR(255) UNIQUE,
phone VARCHAR(20) UNIQUE,
hash VARCHAR(255), -- Argon2/Bcrypt

-- Profile
display_name VARCHAR(100) NOT NULL DEFAULT 'Anonymous',
full_name VARCHAR(255),
bio TEXT,
avatar_url TEXT,

-- Status & Role
status VARCHAR(20) DEFAULT 'OFFLINE',
role VARCHAR(20) DEFAULT 'USER',

-- E2E Keys
public_key TEXT, -- PEM format

-- Settings & Sync
settings JSONB DEFAULT '{}',
contacts_synced_at TIMESTAMPTZ, -- Дата последней синхронизации контактов

-- Soft Delete
deleted_at TIMESTAMPTZ,
expires_at TIMESTAMPTZ, -- 10 лет от deleted_at

created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- contacts.sql (Локальные контакты + маппинг на пользователей)
CREATE TABLE contacts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

  -- Хеш телефона (SHA-256) для синхронизации без передачи номеров
  phone_hash CHAR(64) NOT NULL,

  -- Если пользователь найден в базе
  matched_user_id UUID REFERENCES users(id) ON DELETE SET NULL,

  is_favorite BOOLEAN DEFAULT FALSE,

  created_at TIMESTAMPTZ DEFAULT NOW(),

  UNIQUE(user_id, phone_hash)
);

CREATE INDEX idx_contacts_hash ON contacts(phone_hash);

```

💬 Чаты и Группы

```

-- chats.sql
CREATE TABLE chats (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    type VARCHAR(20) NOT NULL DEFAULT 'PRIVATE',
    -- PRIVATE, GROUP, CHANNEL, NEWS, SERVICE

    -- Content
    title VARCHAR(255),
    avatar_url TEXT,
    description TEXT,

    -- Security (Secret Chats)
    password_hash VARCHAR(255), -- Хеш пароля для "Секретных чатов"

    -- Metadata
    is_deleted BOOLEAN DEFAULT FALSE,
    deleted_at TIMESTAMPTZ,
    expires_at TIMESTAMPTZ, -- +10 лет

    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_chats_type ON chats(type);

-- chat_members.sql
CREATE TABLE chat_members (
    chat_id UUID REFERENCES chats(id) ON DELETE CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,

    joined_at TIMESTAMPTZ DEFAULT NOW(),
    role VARCHAR(20) DEFAULT 'MEMBER', -- ADMIN, MEMBER

    is_muted BOOLEAN DEFAULT FALSE, -- Для каналов
    last_read_message_id UUID, -- Для трекинга непрочитанных

    PRIMARY KEY (chat_id, user_id)
);

-- chat_invites.sql (Коды приглашений)
CREATE TABLE chat_invites (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    chat_id UUID NOT NULL REFERENCES chats(id) ON DELETE CASCADE,

    invite_code VARCHAR(20) UNIQUE NOT NULL,
    used_by_user_id UUID REFERENCES users(id),

    max_uses INT DEFAULT -1, -- -1 = безлимит
    used_count INT DEFAULT 0,

    expires_at TIMESTAMPTZ,

```

```
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

✉ Сообщения и Вложения

```
-- messages.sql
CREATE TABLE messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  chat_id UUID NOT NULL REFERENCES chats(id) ON DELETE CASCADE,
  sender_id UUID NOT NULL REFERENCES users(id) ON DELETE SET NULL,

  type VARCHAR(20) NOT NULL DEFAULT 'TEXT', -- TEXT, VOICE, VIDEO, FILE, IMAGE,
  STICKER

  -- Content
  content TEXT, -- Plain text (для групп/новостей)
  payload TEXT, -- E2E Encrypted Base64 blob

  -- Relations (Replies & Forwards)
  reply_to_id UUID REFERENCES messages(id),
  forwarded_from_user_id UUID,
  forwarded_from_message_id UUID,

  -- Server Metadata
  media_url TEXT,
  thumb_url TEXT, -- Для превью картинок/видео
  file_size INTEGER,
  duration INTEGER,
  mime_type VARCHAR(100),

  -- Soft Delete
  is_deleted BOOLEAN DEFAULT FALSE,
  deleted_at TIMESTAMPTZ,
  expires_at TIMESTAMPTZ, -- +10 лет

  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_messages_chat ON messages(chat_id, created_at DESC);
CREATE INDEX idx_messages_sender ON messages(sender_id);

-- message_attachments.sql (Детальная структура вложений)
CREATE TABLE message_attachments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  message_id UUID REFERENCES messages(id) ON DELETE CASCADE,

  type VARCHAR(20) NOT NULL, -- IMAGE, VIDEO, DOCUMENT, AUDIO, VOICE
  file_url TEXT NOT NULL,
  thumb_url TEXT,
```

```

original_filename VARCHAR(255),
mime_type VARCHAR(100),
file_size INTEGER,
duration INTEGER, -- Сек (для аудио/видео)

uploaded_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_attachments_msg ON message_attachments(message_id);

```

2. CLIENT-SIDE (Storage Matrix)

Стратегия: Безопасно, но просто.

Тип Данных	Хранилище	Описание
Auth Tokens	SecureStorage	JWT + Refresh. AES-256.
E2E Keys	SecureStorage	Приватный ключ. Никогда на сервер.
Contacts	RxDB (IndexedDB)	Локальная база контактов устройства.
Chats/Messages	RxDB (IndexedDB)	Шифрованный WAL. Оффлайн-доступ.
Settings	localStorage	Тема, язык, размер шрифта.

3. ГЛОБАЛЬНЫЕ НАСТРОЙКИ (SETTINGS JSONB)

```

{
  "ui": {
    "theme": "dark",
    "language": "ru",
    "fontSize": "medium",
    "sidebarCollapsed": false,
    "chatTabs": ["all", "personal", "groups", "news"]
  },
  "privacy": {
    "lastSeen": "everyone",
    "profilePhoto": "everyone",
    "allowGroupsByPhone": false,
    "allowCallsByPhone": false
  },
  "notifications": {
    "sound": true,
    "preview": "full",
    "volume": 80,

```

```
"emergencyAlerts": true
},
"security": {
  "twoFactorEnabled": false,
  "lockScreen": true,
  "sessionTimeout": 0
},
"data": {
  "autoDownloadMedia": "wifi",
  "compressImages": true,
  "keepHistory": "forever"
}
}
```

4. ЛОГИКА ХРАНЕНИЯ И УДАЛЕНИЯ (10 Years Rule)

Требование: Данные не удаляются физически сразу.

1. Мягкое удаление (Soft Delete):

- При удалении чата/сообщения/аккаунта устанавливается `deleted_at = NOW()`.
- `is_deleted = TRUE`.
- Данные скрыты из UI.

2. Архивация (Retention):

- `expires_at` вычисляется как `deleted_at + INTERVAL '10 years'`.
- Пример: `expires_at = '2036-06-24'`.

3. Физическое удаление (Cron Job):

- Еженедельный бэкенд-скрипт ищет записи: `WHERE expires_at < NOW()`.
- Физически удаляет строки из таблиц.

END OF DATA SCHEMAS V1.0

2. CLIENT-SIDE (State & Storage)

Стратегия: "Безопасно, но просто".

Инструменты: Zustand (State), RxDB (Offline Cache), SecureStorage (Keys).

2.1 Глобальные Stores (Zustand)

Инструмент: Zustand (легковесная библиотека для управления состоянием).

Назначение: Хранение текущего состояния сессии, открытого чата и UI настроек в памяти.

```
// types/store.ts

// 1. Auth Store (Авторизация)
interface AuthStore {
  token: string | null;
  refreshToken: string | null;
  user: User | null; // Профиль текущего пользователя
  isLoading: boolean;

  login: (data: LoginDto) => Promise<void>;
  logout: () => void;
  refresh: () => Promise<void>;
}

// 2. Chat Store (Состояние чатов)
interface ChatStore {
  activeChatId: string | null; // ID текущего открытого чата
  chatList: ChatSummary[]; // Список чатов (с превью)
  messages: Record<string, Message[]>; // Кэш сообщений по ID чата

  setActiveChat: (id: string) => void;
  addMessage: (chatId: string, msg: Message) => void;
  updateChat: (chatId: string, data: Partial<ChatSummary>) => void;
}

// 3. Settings Store (Настройки интерфейса)
interface SettingsStore {
  theme: 'dark' | 'light' | 'russia';
  language: string;
  sidebarCollapsed: boolean;

  setTheme: (theme: string) => void;
  toggleSidebar: () => void;
}

// 4. Notifications Store (Тосты и алерты)
interface NotificationStore {
  toasts: Toast[];

  addToast: (toast: Toast) => void;
  removeToast: (id: string) => void;
}
```

2.2 Слой Хранения (Storage Matrix)

Тип Данных	Хранилище	Безопасность	Описание
Tokens (JWT)	SecureStorage	● High	AES-256 шифрование на клиенте. Не перезаписывается при перезагрузке.

Тип Данных	Хранилище	Безопасность	Описание
E2E Keys	SecureStorage	● High	Приватный ключ шифрования. Никогда не отправляется на сервер.
UI Settings	localStorage	● Low	Тема, язык, сортировка чатов. Читаемо пользователем.
User Profile	Zustand (RAM)	● Low	Кэш профиля. Сбрасывается при logout.
Messages/Chats	RxDB (IndexedDB)	● Medium	Шифруется через WAL (Write-Ahead Logging) или PouchDB encryption plugin.
Drafts	localStorage	● Low	Черновики сообщений.

2.2 RxDB Schema (Mobile/Web Offline)

Для синхронизации с сервером используется RxDB. Схема локальной БД упрощена для скорости.

```
import { newRxDB } from 'rxdb';
import { getRxStorageDexie } from 'rxdb/plugins/storage-dexie';

// Локальная схема сообщений (содержимое может быть зашифровано)
export const messageSchema = {
  version: 0,
  primaryKey: 'id',
  type: 'object',
  properties: {
    id: { type: 'string' },
    chatId: { type: 'string' },
    senderId: { type: 'string' },
    encryptedPayload: { type: 'string' }, // Base64 blob
    type: { type: 'string' },
    timestamp: { type: 'number' }
  },
  required: ['id', 'chatId', 'timestamp'],
  indexes: ['chatId', 'timestamp'] // Для быстрого скролла
};

export const chatListSchema = {
  version: 0,
  primaryKey: 'id',
  type: 'object',
  properties: {
    id: { type: 'string' },
    lastMessageText: { type: 'string' }, // Decrypted preview
    unreadCount: { type: 'number' },
    updatedAt: { type: 'number' }
  }
};
```

2.3 Secure Storage (Реализация)

Используем библиотеку `secure-local-storage` (для Web) или `expo-secure-store` (Mobile).

```
import { getStorage } from 'secure-local-storage';
import * as Crypto from 'expo-crypto'; // Mobile
// или window.crypto.subtle для Web

const storage = getStorage({
  encrypt: true, // AES-GCM 256-bit
  key: 'balloo-app-key' // В реальности лучше derive из PIN или биометрии
});

// Сохранение токена
await storage.setItem('authToken', jwtToken);

// Получение ключа E2E
const e2eKey = await storage.getItem('user_e2e_private_key');
```

3. ГЛОБАЛЬНЫЕ НАСТРОЙКИ (SETTINGS SCHEMA)

Расположение: `user.settings` (JSONB на сервере), `uiStore` (Zustand на клиенте).

3.1 Структура (TypeScript Interface)

```
interface GlobalSettings {
  // UI / Интерфейс
  ui: {
    theme: 'dark' | 'light' | 'russia' | 'custom';
    language: 'ru' | 'en' | 'tt' | ...; // 12 языков
    fontSize: 'small' | 'medium' | 'large';
    sidebarCollapsed: boolean;
    showPreviews: boolean;
    dateFormat: 'DD.MM.YYYY' | 'MM/DD/YYYY';
  };

  // Privacy / Приватность
  privacy: {
    lastSeen: 'everyone' | 'contacts' | 'nobody';
    onlineStatus: 'everyone' | 'contacts' | 'nobody';
    profilePhoto: 'everyone' | 'contacts' | 'nobody';
    allowGroupsByPhone: boolean;
    allowCallsByPhone: boolean;
  };

  // Notifications / Уведомления
  notifications: {
```

```

    sound: boolean;
    push: boolean;
    desktop: boolean;
    preview: 'full' | 'hidden' | 'off';
    volume: number; // 0-100
    emergencyAlerts: boolean; // Вкл/выкл экстренные оповещения
};

// Security / Безопасность
security: {
    twoFactorEnabled: boolean;
    lockScreen: boolean;
    sessionTimeout: number; // minutes (0 = infinite)
    trustedDevices: string[]; // Device IDs
};

// Data / Данные
data: {
    autoDownloadMedia: 'wifi' | 'wifi_mobile' | 'off';
    compressImages: boolean;
    keepHistory: 'forever' | '30_days' | '3_months';
};
}

```

3.2 Значения по умолчанию (Default Settings)

```

{
  "ui": {
    "theme": "dark",
    "language": "ru",
    "fontSize": "medium",
    "sidebarCollapsed": false,
    "showPreviews": true,
    "dateFormat": "DD.MM.YYYY"
  },
  "privacy": {
    "lastSeen": "everyone",
    "onlineStatus": "everyone",
    "profilePhoto": "everyone",
    "allowGroupsByPhone": false,
    "allowCallsByPhone": false
  },
  "notifications": {
    "sound": true,
    "push": true,
    "desktop": true,
    "preview": "full",
    "volume": 80,
    "emergencyAlerts": true
  },
}

```

```

"security": {
  "twoFactorEnabled": false,
  "lockScreen": true,
  "sessionTimeout": 0,
  "trustedDevices": []
},
"data": {
  "autoDownloadMedia": "wifi",
  "compressImages": true,
  "keepHistory": "forever"
}
}

```

4. СИНХРОНИЗАЦИЯ (SYNC STRATEGY)

Проблема: Как синхронизировать **RxDB** (Client) и **PostgreSQL** (Server)?

Решение: Conflict-free Replicated Data Types (CRDTs) или Last-Write-Wins (LWW).

1. Сообщения:

- Отправка: **RxDB** -> **WebSocket** -> **PostgreSQL**.
- Получение: **PostgreSQL** -> **WebSocket** -> **RxDB**.
- Конфликт: Невозможен (сообщения immutable).

2. Профиль/Настройки:

- Механизм **LWW (Last Write Wins)**.
- При обновлении settings на клиенте -> отправляем PATCH на сервер.
- При логине -> скачиваем **user.settings** и перезаписываем локальные.

3. Аватарки:

- Клиент загружает файл на Яндекс.Диск.
- Получает URL.
- Отправляет URL на сервер (PATCH /api/users/me/avatar).
- Сервер обновляет **User.avatar**.

5. ХРАНЕНИЕ ФАЙЛОВ (MEDIA STORAGE)

Решение: Yandex Disk SDK (объектное хранилище S3-совместимое).

Тип	Папка в бакете	Логика
Аватары	/avatars/{userId}	Переписываются при смене. Max 5MB.
Фото (Stories)	/stories/{userId}/{timestamp}	Удаляются через 24ч (Cron job).

Тип	Папка в бакете	Логика
Файлы (Чаты)	/files/{chatId}/{uuid}	Сохраняются вечно (или по keepHistory).
Видео	/videos/{chatId}/{uuid}	Компрессия на клиенте перед загрузкой.

END OF DATA SCHEMAS